



SAMUELI SCHOOL OF ENGINEERING
UNIVERSITY OF CALIFORNIA, LOS ANGELES

CAPSTONE REPORT - SUMMER 2025

**Predictive Maintenance Using Machine
Learning Time Series Model**
GitHub Repository

Li-Chun Huang, Grace Cheng, Portia Huang, Yen-Yun Kuo

supervised by
Ramon Millan, MN8 Energy
ramon.millan@mn8.com

Abstract

The rapid growth of solar photovoltaic (PV) systems has increased the need for reliable predictive maintenance strategies to minimize downtime and energy loss. Inverters, as critical components in PV plants, are prone to failures caused by thermal stress, environmental fluctuations, and grid instability. This project develops a predictive maintenance framework leveraging a hybrid deep learning model to forecast inverter breakdowns using multivariate time-series data from MN8 Energy. A comprehensive data pipeline was constructed, including cleaning, missing-value imputation, device-wise standardization, feature engineering for thermal and voltage stress, and failure session labeling with a five-day prediction horizon. A CNN-LSTM architecture was then trained with focal loss to address severe class imbalance, capturing both short-term fluctuations and long-term degradation patterns. Experimental results show that the model achieves strong discriminative performance as ROC AUC 0.9380 on training classification, 0.9535 on validation, and 0.6908 on test classification. The model is capable of providing early-warning signals up to five days in advance, though precision remains limited due to data imbalance and temporal shifts. Post-processing with temporal smoothing improved alert usability for operational deployment. The study demonstrates the potential of deep learning to enhance proactive maintenance of large-scale PV assets and highlights future opportunities to integrate exogenous data (e.g., season, weather) and explore time-series based models for greater robustness and interpretability.

Keywords: Solar inverter, failure prediction, time-series model, CNN-LSTM, Anomaly Detection, AUCPR, deep learning, renewable energy, preventive maintenance, predictive modeling

Contents

1. Introduction	5
2. Literature Review	6
2.1. Working Flow and Metrics of Solar-Based Inverter	6
2.2. Time Series Forecasting	8
2.3. Traditional Time Series Method(ARIMA,SES)	8
2.4. Deep Learning Network for Time Series	9
3. Problem Define	13
3.1. Data Structure	13
3.2. Task	14
4. Methodology	15
4.1. Data Transformation	15
4.2. Identify Failure Sessions	16
4.3. Data Preprocessing	17
4.4. Dataset Construction	23
4.5. Model Architecture	24
4.6. Training Procedure	25
5. Analysis	27
5.1. Evaluation methodology	27
5.2. Score calibration and thresholding	27
5.3. Main results	28
5.4. Post-processing methodology	31
5.5. Effect of temporal smoothing	31
6. Conclusion and Future Work	33
6.1. Conclusion	33
6.2. Limitations and Future Work	34
7. Acknowledgments	35

A. Appendix	38
A.1. Raw Data	38
A.2. Metrics List	38

1. Introduction

The rapid expansion of renewable energy has brought solar photovoltaic (PV) systems to the forefront of global decarbonization strategies. Large-scale PV plants now play a critical role in supplying clean electricity, but their efficiency and reliability depend heavily on the performance of key components such as inverters. Inverters, which convert DC power from solar panels into usable AC power, are particularly vulnerable to failures due to fluctuating environmental conditions, thermal stress, and grid-side instabilities. Unexpected breakdowns can result in costly downtime, reduced energy yield, and increased maintenance expenses. As a result, the ability to predict inverter failures before they occur has become a crucial research and operational priority for improving system reliability and optimizing maintenance strategies in the solar industry.

Time series forecasting provides a valuable approach for addressing this challenge, as inverter data is recorded continuously through sensor measurements such as voltages, currents, power flows, and internal temperatures. Traditional methods, including ARIMA and exponential smoothing, have offered early insights into temporal patterns but struggle to capture nonlinear relationships or long-range dependencies inherent in PV operations. Recent advances in machine learning and deep learning have significantly expanded the toolkit for predictive maintenance, enabling the modeling of complex patterns and interactions within large-scale operational datasets. These methods not only improve forecast accuracy but also allow for anomaly detection and early warning systems that can directly reduce unplanned outages.

A hybrid deep learning model combining convolutional neural networks (CNNs) and long short-term memory (LSTM) networks is developed and evaluated for inverter failure prediction. Within this framework, the CNN component is employed to extract local temporal and multivariate patterns across sensor signals, while the LSTM component is designed to capture long-term dependencies and sequential dynamics that are critical for anticipating progressive degradation. By integrating these complementary capabilities, the dual-layer architecture provides a more robust predictive capacity than either model individually. Applied to MN8 Energy’s historical inverter dataset, this approach demonstrates the potential of deep learning to support proactive maintenance planning and to enhance the operational resilience of large-scale solar PV portfolios.

2. Literature Review

2.1. Working Flow and Metrics of Solar-Based Inverter

Grid-connected DC-coupled hybrid photovoltaic (PV)–battery systems connect both the PV array and the battery to a shared DC bus within a hybrid inverter, with a bidirectional DC–DC converter enabling battery charge and discharge. This topology requires only one DC–AC conversion stage for grid export, improving round-trip efficiency compared to AC-coupled designs, which perform multiple AC↔DC conversions [1][2]. A key operational advantage is the ability to recover energy that would otherwise be lost due to inverter clipping: surplus PV output above the inverter’s AC rating can be stored directly in the battery [3]. DC-coupled systems also integrate conversion hardware, reducing balance-of-system components and potentially lowering the levelized cost of energy in certain configurations [4].

Figure 1 illustrates the basic configuration of such a system. In this architecture, both the PV array and the battery are connected to the hybrid inverter’s DC bus, allowing direct DC-to-DC energy exchange between the two sources. The inverter then converts DC power into AC for delivery to household loads, backup circuits, or export to the utility grid via the switchboard. This arrangement minimizes conversion stages—typically requiring only one DC–AC stage for grid export—thereby improving efficiency and enabling direct storage of excess PV generation for clipping recovery.

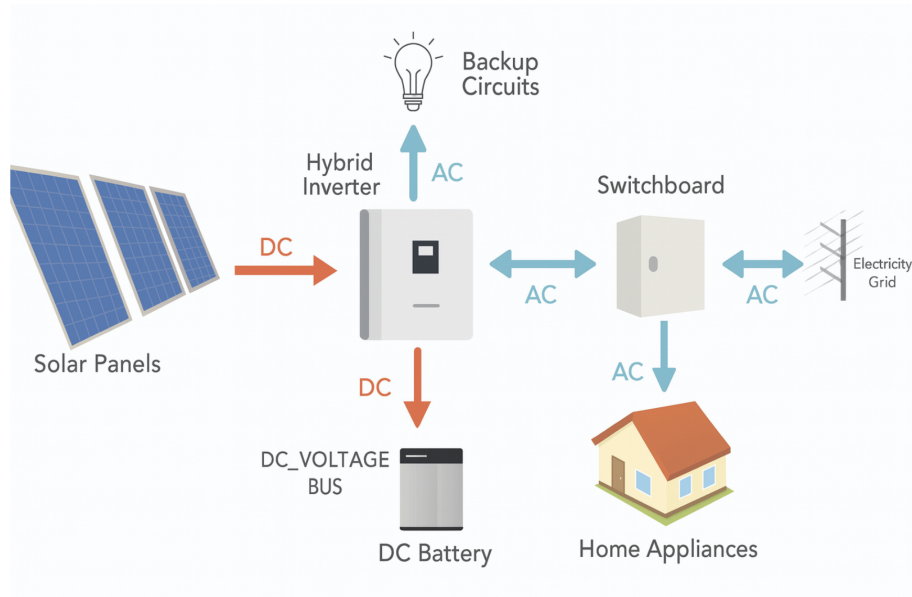


Figure 1: Basic layout diagram of a grid-connected DC-coupled hybrid solar inverter with DC battery system.

However, the shared inverter capacity introduces trade-offs in simultaneous PV generation, battery discharge, and grid services, requiring coordinated dispatch and power sharing strategies [5]. Reliability studies indicate DC- and AC-coupled architectures differ in failure modes and maintenance demands, influencing design choice based on redundancy requirements [2][5]. In DC-coupled systems, maximum power point tracking (MPPT) and battery DC–DC control must be integrated to optimize PV generation, storage, and grid delivery under variable irradiance and market conditions. Advanced MPPT and robust control methods, including sliding-mode strategies, have been proposed to maintain stability during rapid irradiance fluctuations [6]. The dataset analyzed in this study, which includes both DC-side variables (e.g., DC bus voltage) and AC-side parameters (e.g., three-phase voltages), aligns with the telemetry structure of DC-coupled hybrid inverters as described in these works.

Building upon the understanding of the overall workflow of a solar inverter, it becomes clear that monitoring key electrical parameters—specifically AC voltage, DC voltage, and AC power—is essential. These parameters reflect how efficiently the inverter is converting and delivering energy, and they serve as primary indicators of its operational health. For instance, a sudden drop in AC power or a sustained period where AC power remains at zero may signal issues such as equipment degradation, inverter shutdowns, or grid disconnection. Similarly, irregularities in AC or DC voltage—such as unexpected spikes or drops—can indicate electrical instability or hardware malfunction. Continuous, real-time monitoring of these signals helps identify deviations from normal performance patterns, enabling early detection of anomalies before they escalate into critical failures. This is especially important in distributed solar systems, where downtime from a single inverter can significantly impact overall system output.

Reliable inverter failure prediction requires consideration of factors beyond simple power metrics. Internal and external temperatures, including component-level heating and ambient conditions, directly affect inverter efficiency and safety mechanisms, often leading to thermal derating, shutdowns, or long-term degradation. Seasonal variation also plays an important role, as fluctuations in solar irradiance, weather patterns, and ambient temperature across different months alter the inverter’s workload and stress levels. Similarly, the time of day introduces nonlinear yet predictable dynamics, with rapid changes in irradiance from sunrise to noon to evening shaping the output profile. In addition, unit-specific characteristics—such as localized shading, hardware differences, or maintenance history—can cause inverters to respond differently even under seemingly identical operating conditions. Integrating these performance, environmental, and con-

textual variables therefore enables the development of more holistic and accurate fault prediction models.

2.2. Time Series Forecasting

Time series forecasting involves using historical data points, recorded sequentially over time, to predict future values. In energy systems such as solar PV plants, time series models can capture patterns like trends, seasonality, and short-term fluctuations in variables such as irradiance, temperature, and power output, enabling both generation prediction and anomaly detection. Classical approaches, such as autoregressive integrated moving average (ARIMA), have been widely used for their interpretability and suitability for stationary data [7]. More recently, machine learning and deep learning methods—particularly recurrent neural networks (RNN) and long short-term memory (LSTM) architectures—have shown improved accuracy in modeling non-linear dependencies and long-range temporal patterns in PV forecasting tasks [8].

2.3. Traditional Time Series Method(ARIMA,SES)

Among traditional approaches, the Autoregressive Integrated Moving Average (ARIMA) model has been one of the most established and widely applied methods for time series forecasting. It integrates three components—autoregression (AR), differencing for stationarity (I), and moving averages (MA)—to model linear dependencies within sequential data [9]. The method’s structured framework allows practitioners to systematically identify parameters using the Box–Jenkins methodology, making ARIMA particularly effective for univariate datasets with stable seasonal and trend patterns. In the context of renewable energy systems, ARIMA has been used to forecast solar irradiance, PV power output, and load demand due to its balance of accuracy, interpretability, and computational efficiency [10]. However, ARIMA’s reliance on linearity assumptions and its sensitivity to sudden, nonlinear fluctuations—such as rapid cloud cover changes—can limit its predictive performance in highly variable PV environments.

Simple Exponential Smoothing (SES) offers another well-established statistical approach, applying exponentially decreasing weights to past observations so that more recent values exert a stronger influence on forecasts [11]. SES is well-suited for short-term predictions without strong seasonal or trend components, and it remains popular as a lightweight baseline model in solar PV forecasting tasks where computational resources are limited. Its strength lies in responsiveness to recent data while avoiding overfitting to noise; however, like ARIMA, it struggles with complex

nonlinearities or long-term dependencies inherent in many energy systems.

ARIMA and SES provide a robust starting point for time series forecasting in solar applications. They are often used as benchmarks against which more sophisticated machine learning and deep learning models—such as recurrent neural networks (RNN) and long short-term memory (LSTM) architectures—are evaluated [12]. While modern neural networks can model nonlinear dynamics and long-range temporal patterns more effectively, the transparency, reproducibility, and domain acceptance of these traditional models continue to make them valuable tools in the early stages of PV forecasting system design.

2.4. Deep Learning Network for Time Series

Deep learning is a subfield of machine learning that focuses on training multilayered artificial neural networks to automatically extract hierarchical feature representations from data. Unlike traditional models that often depend on handcrafted features, deep learning networks learn directly from raw inputs, progressively transforming them into increasingly abstract and informative representations [13]. This approach allows them to model highly nonlinear relationships and interactions, making them suitable for complex datasets in fields such as image recognition, natural language processing, and speech analysis. The scalability and adaptability of these models have made deep learning one of the most influential developments in modern artificial intelligence.

The same principles that make deep learning powerful in vision and language also apply to time-dependent data. Time series forecasting often requires capturing both short-term variations and long-range dependencies, patterns that may be subtle, nonlinear, and intertwined with external factors. Deep learning architectures are uniquely capable of addressing these challenges by learning sequential dependencies, detecting hidden structures, and generalizing across different temporal contexts [14]. In energy systems, finance, and climate modeling, deep learning has increasingly been adopted as an alternative to traditional statistical methods, offering improvements in accuracy and robustness. This makes it a natural extension of classical forecasting techniques, setting the stage for more specialized architectures such as recurrent neural networks (RNN), long short-term memory (LSTM) networks, and transformers.

2.4.1. AutoEncoder

Autoencoders, composed of encoder–decoder neural network structures, have become an influential tool in time-series forecasting and anomaly detection because of their ability to compress

sequential data into compact latent representations and then reconstruct it. In energy-related applications, this reconstruction framework allows researchers not only to detect abnormal fluctuations in solar photovoltaic (PV) data but also to generate meaningful features that improve forecasting accuracy. [14] applied a deep autoencoder to PV power time-series data and showed that even with missing or insufficient training data, the model was able to identify anomalies effectively, reinforcing its robustness in real-world deployment. Similarly, [15] integrated a convolutional autoencoder with principal component analysis (PCA) to preprocess large-scale PV datasets. Their results demonstrated that filtering out anomalous input sequences could improve forecasting accuracy by more than twenty percent across thousands of PV sites, highlighting the value of autoencoder-based anomaly detection as a data-cleaning mechanism for time-series prediction. Extending this line of research, [16] employed a variational autoencoder (VAE) framework to tackle short-term solar power forecasting and reported consistent performance gains over both traditional statistical methods and advanced deep learning models such as recurrent neural networks and gated recurrent units.

2.4.2. CNN

The versatility of Convolutional Neural Networks (CNNs) extends far beyond image recognition; they have emerged as effective tools for time-series forecasting by leveraging their ability to detect local temporal patterns and hierarchies. A recent study on solar irradiance forecasting by [17] introduced a CNN–attention–LSTM hybrid that demonstrated how CNNs excel at extracting spatial features (e.g., from sky images), while LSTM captures temporal dependencies, resulting in significantly improved performance over traditional methods. Similarly, [18] showcased a CNN–LSTM structure tailored for PV power output forecasting, where the CNN component first classifies weather conditions—such as sunny or cloudy days—and the LSTM then models power generation patterns specific to those classifications. This approach achieved mean absolute percentage errors as low as 4.58% on sunny days, underscoring CNNs’ efficacy in enhancing model adaptability to environmental context .

On a broader methodological level, [19] highlighted that CNNs can outperform RNN-based approaches in various time-series scenarios by incorporating advanced techniques like dilated convolutions and smoothing preprocessing. These enhancements allow CNN models to effectively capture both short-term patterns and extended dependencies, while maintaining computational efficiency. CNNs’ structured weight sharing and hierarchical feature extraction not only deliver

faster training and inference but also demonstrate competitive performance across diverse forecasting tasks, making them a compelling alternative to more traditional sequential models like RNNs or LSTMs.

2.4.3. RNN/LSTM

Recurrent Neural Networks (RNNs) were among the first deep learning architectures explicitly designed for sequential data, making them a natural choice for time-series forecasting. Their feedback connections allow information to persist across time steps, enabling the capture of temporal dependencies in sequential datasets [20]. However, traditional RNNs often suffer from vanishing and exploding gradient problems, which limit their ability to model long-term dependencies effectively [21].

To address these limitations, Long Short-Term Memory (LSTM) networks were introduced by [22]. By incorporating memory cells and gating mechanisms, LSTMs regulate the flow of information, allowing the model to retain relevant patterns over long horizons while filtering out noise. This design has made LSTMs one of the most widely applied architectures in solar energy forecasting. For example, [23] demonstrated that LSTM-based models significantly outperformed both ARIMA and standard feedforward neural networks in predicting short-term PV output, particularly under rapidly changing weather conditions. Similarly, [24] applied LSTMs to renewable energy forecasting and found substantial gains in accuracy compared to traditional statistical approaches, highlighting their strength in handling nonlinear and nonstationary dynamics.

Together, RNNs and LSTMs represent a critical progression in deep learning for time-series tasks. While CNNs excel at local feature extraction, LSTMs provide the ability to model long-range temporal dependencies, making them especially powerful when integrated into hybrid frameworks for energy forecasting. Their adaptability across contexts—ranging from power load prediction to solar irradiance estimation—has established LSTMs as a benchmark against which newer models, such as GRUs and Transformers, are frequently compared.

2.4.4. GRU

Gated Recurrent Units (GRUs) were introduced as a streamlined alternative to LSTMs, designed to address the same vanishing gradient issues while reducing computational complexity. Unlike LSTMs, which maintain separate memory cells and three gating mechanisms (input, forget, and output), GRUs merge the memory and hidden state into a single representation and

rely on only two gates—the update and reset gates [25]. This simplification makes GRUs computationally more efficient while still retaining the ability to capture long-term dependencies in sequential data.

In the context of time-series forecasting, GRUs have been shown to deliver performance comparable to, and in some cases surpassing, LSTMs, particularly when training data are limited or when faster convergence is needed. For instance, [26] applied GRUs to short-term wind speed forecasting and found that the model achieved higher predictive accuracy than LSTMs while requiring less training time. Similarly, in renewable energy forecasting, GRUs have proven effective at modeling nonlinear temporal dynamics while maintaining lower parameter counts, which is advantageous for real-time applications on embedded systems or edge devices [27].

By balancing efficiency with predictive power, GRUs have become a strong candidate for solar PV forecasting tasks, especially in hybrid or ensemble models where computational speed is a practical constraint. Their adoption highlights the ongoing trend toward architectures that combine temporal modeling capacity with reduced model complexity, ensuring both scalability and adaptability in energy forecasting systems.

2.4.5. Transformer

Transformer architectures, originally developed for natural language processing, have recently emerged as powerful tools for time-series forecasting. Their defining feature is the self-attention mechanism, which allows the model to weigh the importance of different time steps in the input sequence without relying on sequential recurrence. This enables Transformers to capture both short- and long-range dependencies more efficiently than RNN-based models [28]. The parallelizable structure of Transformers also reduces training time, making them highly scalable for large datasets commonly found in energy systems.

In renewable energy forecasting, Transformer variants have shown strong performance in predicting solar irradiance and power output. For example, [29] introduced the Informer model, which modifies the attention mechanism to handle long input sequences more efficiently, and demonstrated significant improvements in accuracy and computation time for energy and weather time series. Similarly, [30] proposed the Temporal Fusion Transformer (TFT), which combines attention with recurrent components to provide both high predictive accuracy and interpretability. These innovations highlight the Transformer’s flexibility in modeling temporal patterns that are complex, nonlinear, and seasonally variable—making it particularly well-suited for photovoltaic

forecasting.

Given the strengths and limitations of the above approaches, this study adopts a hybrid CNN–LSTM architecture as the forecasting model. CNNs are effective at extracting local temporal patterns and reducing noise in sequential data through convolutional filters, while LSTMs excel at modeling long-term dependencies and sequential dynamics. By stacking these two layers, the hybrid design leverages CNN’s feature extraction capabilities alongside LSTM’s memory mechanism, producing a more robust representation of complex photovoltaic time series. Such integration has been shown to improve accuracy in renewable energy forecasting tasks where both short-term fluctuations and long-range seasonal trends coexist [31]. The choice of this dual-layer structure thus reflects a balance between predictive performance and model interpretability, making it well-suited for anticipating inverter behavior and supporting proactive maintenance strategies in solar power systems.

3. Problem Define

3.1. Data Structure

Table 16 summarizes the structure of the raw dataset used in this study. The data is organized as a multi-source, long-format time series consisting of over 3.3 million rows and 12 columns. Each row corresponds to a unique observation, identified by device-specific information—`device_id`, `device_name`, `device_type`, and `site_id`—along with precise timestamps (`event_utc_time`, `event_local_time`).

The core measurements are recorded through the `metric` column, which indicates the type of parameter being captured (e.g., physical, electrical, or operational), and the `value` column, which contains the corresponding numeric or categorical reading. This schema supports scalable monitoring of diverse device behaviors. However, for model training, the data must be pivoted into a wide-format time series matrix, where each metric becomes a distinct feature aligned along the temporal axis.

The metrics of raw data can be categorized into current, power, voltage, energy, energy loss, status, performance, etc. The richness of the metric categories highlights the complexity of inverter behavior. Electrical signals include AC and DC currents, voltages, and power under measured, expected, or regression-estimated conditions. Performance metrics capture energy delivered, received, or lost due to shading, curtailments, or outages, while status indicators provide discrete

information on warnings, internal temperature, fault words, and anomalous flags. Together, these heterogeneous signals span continuous, binary, and categorical domains, reflecting both physical processes and operational diagnostics. Such diversity is valuable for modeling inverter health trajectories but complicates preprocessing, as scaling, encoding, and alignment must account for differences in physical units, value ranges, and data frequencies.

A further complication arises from the temporal and contextual dependencies within the data. Inverter outputs are inherently shaped by exogenous factors such as irradiance patterns, seasonal weather, and site-specific conditions, while endogenous factors like component temperature or internal pressure contribute to degradation dynamics. Missing values, measurement errors (e.g., abnormally high or invalid readings), and imbalanced labels between normal operation and rare failure events add to the challenge. Addressing these issues requires careful data cleaning, anomaly handling, and feature selection to construct an input space that preserves the predictive signals most relevant to impending faults. By formalizing the dataset into a structured, high-quality representation, the foundation is set for robust modeling with hybrid architectures such as CNN-LSTM networks.

3.2. Task

The first stage of the study involves systematic data preprocessing to ensure the integrity and reliability of the input signals. This includes the removal of missing values and the correction of erroneous sensor readings, such as abnormally high temperatures or voltages that represent noise rather than true operating conditions. Following data cleaning, inverter failure sessions are identified and used to construct labeled sequences: for each detected failure, the preceding five days of operational data are marked as the failure prediction window. This labeling strategy allows the model to learn precursors of degradation rather than only post-failure behavior.

Once the labeled dataset is established, feature selection is performed to incorporate the most relevant operational and environmental variables, ensuring that both performance metrics and contextual influences are represented. The hybrid CNN-LSTM model is then trained on this dataset to forecast inverter failures. Special attention is given to the class imbalance inherent in the data, where the majority of sessions correspond to normal operation. To address this, performance evaluation emphasizes not only accuracy—which can be misleading in imbalanced settings—but also validation loss trends and area under the precision-recall curve (AUCPR), both of which provide more robust insight into the model’s discriminative ability. Ensuring that

these curves follow expected patterns confirms that the model is capturing meaningful temporal features rather than defaulting to trivial predictions.

4. Methodology

4.1. Data Transformation

To predict the occurrence of inverter breakdowns, we focus exclusively on the data collected from inverters, as it provides the most direct and informative signals. The raw dataset was first processed in **Azure Data Factory**, where we filtered out records from non-inverter devices and retained only inverter-related measurements. Subsequently, the data was pivoted on individual metrics, restructuring the dataset into a wide format that is more suitable for analysis and model development. This transformation facilitates the construction of predictive models by aligning measurements consistently across time. The transformed dataset contains a total of 57 inverter-related metrics, covering electrical currents, voltages, power, energy statistics, and operational status indicators. Table 1 presents several representative features along with their descriptions. A complete list of all features is provided in Appendix A.2 for reference.

Table 1: Features Selected for Modeling

Feature Name	Description	Unit	Example Value
event_local_time	Local timestamp of the measurement (5-min interval)	datetime	2021-12-02 00:00:00
device_name	Unique identifier of the inverter	categorical	INV 51
AC_MOD_ADMISSION_TEMP	Ambient module admission temperature	°C	35.0
INTERNAL_TEMP	Internal inverter temperature	°C	40.0
AC_VOLTAGE_AB	AC line voltage between phase A and B	V	33800.0
AC_VOLTAGE_BC	AC line voltage between phase B and C	V	33700.0
AC_VOLTAGE_CA	AC line voltage between phase C and A	V	33900.0
DC_VOLTAGE	DC bus voltage	V	1000.0
AC_POWER	Active power output of inverter	W	120000.0

4.2. Identify Failure Sessions

4.2.1. Definition and Rationale

We define a **failure session** as a contiguous time interval during which an inverter produces zero active power while the plant as a whole remains in operation. Since AC power is the most direct indicator of inverter functioning, it is adopted as the primary signal for session detection.

Let $P_i(t)$ denote the AC power of inverter i at timestamp t , and $P_{\text{plant}}(t)$ denote the aggregated plant power. A failure session for inverter i is any maximal interval $[t_s, t_e]$ such that

$$P_i(t) = 0 \quad \text{and} \quad P_{\text{plant}}(t) > 0, \quad \forall t \in [t_s, t_e]. \quad (1)$$

For robustness, missing values in $P_i(t)$ are treated as 0, meaning they are interpreted as *non-generation events* (e.g., inverter shutdown) or *communication losses*.

4.2.2. Robustness Improvement

To reduce false positives and noise, the following constraints are applied:

- **Minimum Duration:**

Short-term interruptions may occur due to communication loss or external factors such as shading. To filter out such spurious cases, we define a minimum session length $\Delta_{\text{min}} = 3$ days and remove any detected failure session that does not satisfy

$$(t_e - t_s) \geq \Delta_{\text{min}}. \quad (2)$$

- **Maintenance Masking:**

We further analyze abnormal patterns of sessions across all inverters. If multiple inverters shut down and recover simultaneously or within a short time window, such sessions are more likely caused by scheduled maintenance rather than true failures. These sessions are therefore labeled as *maintenance sessions* and excluded from failure analysis.

4.3. Data Preprocessing

4.3.1. Feature Selection

The first step of data preprocessing is to select useful features from the 57 original inverter-related metrics. In consultation with domain experts from Mn8, we focused on a subset of variables that are most relevant to inverter performance and potential breakdowns. Specifically, we retained features related to AC power, AC voltage, DC voltage, and temperature measurements, as these variables are directly linked to the electrical and thermal conditions of inverter operation.

The rationale for this selection is twofold. First, power and voltage variables provide fundamental information on the operational status of the inverter and its interaction with the grid. Second, temperature-related metrics capture the thermal stress of the device, which is a critical factor for equipment reliability and failure prediction. By narrowing down to this focused set of features, we reduce noise from less informative variables while preserving the essential signals for subsequent modeling.

4.3.2. Outlier Handling

We first analyzed the dataset and identified anomalies in the variable `STATUS_AC_MOD_ADMISSION_TEMP`. As shown in Figure 2, some temperature readings exceeded 100°C , which is not physically reasonable for inverter operation. These anomalous values were therefore filtered and replaced with NaN, to be handled in the subsequent preprocessing steps.

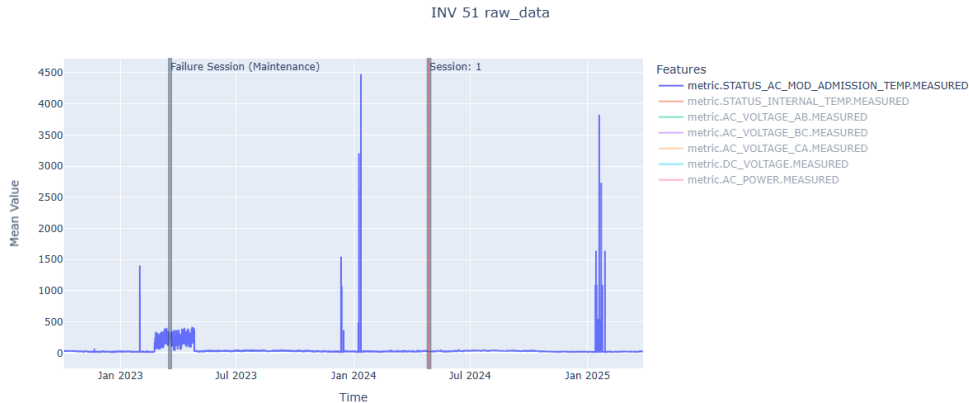


Figure 2: Example of anomalous temperature readings in `STATUS_AC_MOD_ADMISSION_TEMP`, with values exceeding 100°C .

4.3.3. Missing Value Imputation

Missing values frequently occur due to unstable connections or temporary sensor failures, and are also related to the potential occurrence of future failure sessions. Therefore, it is important to impute these missing values for modeling while preserving the underlying distribution of missingness.

In this step, we apply different strategies depending on the duration of missing intervals:

- **Short-term gaps (≤ 30 minutes):** When the duration of missing values is within 30 minutes (i.e., up to six consecutive 5-minute intervals), we apply *linear interpolation* to reconstruct the missing signal smoothly.
- **Long-term missing values (> 30 minutes):** For extended periods of missing data, which may indicate prolonged inverter breakdowns or communication losses, we fill the missing entries with a constant value of 0.

To preserve the distribution of missingness information, we add an additional binary indicator feature corresponding to each imputed variable. For example, along with `STATUS_AC_MOD_ADMISSION_TEMP`, we introduce `STATUS_AC_MOD_ADMISSION_TEMP_missing`. The indicator takes the value `True` if the original record was missing and `False` otherwise. In this way, we retain both the missing distribution and the missing rate in the dataset.

4.3.4. Downsampling

Our task is to predict failure sessions on a daily time frame. In this setting, several days of input data may be required to make a reasonable prediction. Using too short an input window may fail to capture sufficient temporal patterns, whereas too long an input may introduce noise and reduce model robustness.

However, the raw inverter data is recorded at 5-minute intervals. Directly using this fine-grained resolution would result in extremely large input sequences and a high level of noise. To reduce noise and computational complexity, we downsampled the data to a 30-minute resolution. Different aggregation strategies were applied depending on the semantic meaning of the variables:

- **Voltage and Power Metrics:** Aggregated using the *mean* to capture the overall trend of the signal within each 30-minute window.

- **Temperature Metrics:** Aggregated using the *maximum*, since peak temperature is more relevant for identifying thermal stress that may cause inverter breakdowns.
- **Missing Indicators:** Aggregated using the *mean*, which corresponds to the proportion of missing values within each window.

The original dataset contained 6,126,272 rows at 5-minute resolution, which was reduced by a factor of six after downsampling to 30-minute intervals. The detailed aggregation rules are summarized in Table 2.

Table 2: Downsampling strategies applied to different metrics.

Metric	Aggregation Strategy
AC_VOLTAGE_AB.MEASURED	mean
AC_VOLTAGE_BC.MEASURED	mean
AC_VOLTAGE_CA.MEASURED	mean
DC_VOLTAGE.MEASURED	mean
AC_POWER.MEASURED	mean
STATUS_AC_MOD_ADMISSION_TEMP.MEASURED	max
STATUS_INTERNAL_TEMP.MEASURED	max
*_missing (all missing indicators)	mean

4.3.5. Data Cleaning

During data exploration, we identified two major anomalous intervals in the dataset. First, abnormal values were observed between February 23, 2023 and August 26, 2023 (see Figure 3). Second, we found that the data prior to December 23, 2021 exhibited inconsistent frequency patterns (see Figure 4). These anomalies may have been caused by sensor malfunctions, software errors, or distribution conflicts.

To ensure the reliability of the dataset, such anomalous intervals were removed from the analysis. This step prevents spurious signals from contaminating the training data and improves the robustness of subsequent modeling tasks.

4.3.6. Feature Engineering

In addition to the selected features described above, we engineered additional features to help the model identify irregular patterns.

- **Sin/Cos Encoded Time:**

Since solar power generation is highly dependent on sunlight and seasonal variations, the in-

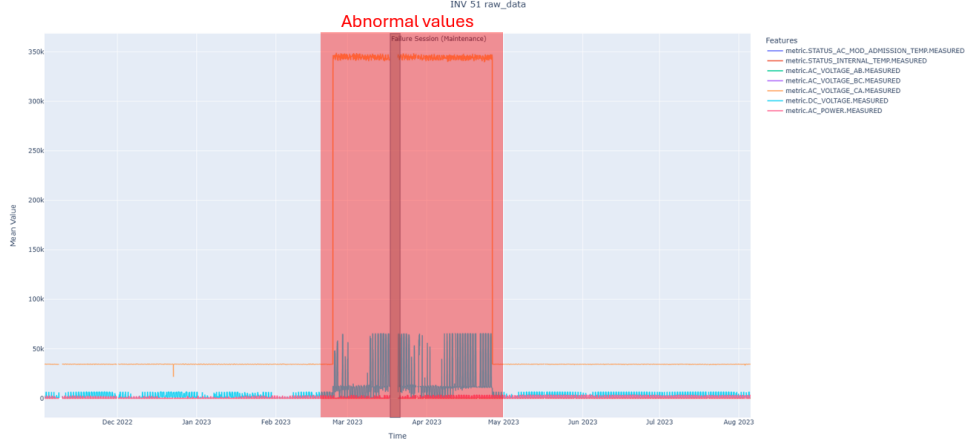


Figure 3: Distribution of abnormal values detected between February 23, 2023 and August 26, 2023. The highlighted red region indicates the anomalous interval that was excluded from subsequent analysis.



Figure 4: Inconsistent frequency patterns observed in the data prior to December 23, 2021.

verter metrics exhibit strong daily and yearly periodicity. To capture such cyclical patterns, we introduced time-related features using sinusoidal encoding. This representation avoids discontinuities at the boundaries (e.g., 23:00 and 0:00) and preserves the cyclic nature of time.

Specifically, we added two features, `hour_sin` and `hour_cos`, to encode the hour of the day, and two features, `month_sin` and `month_cos`, to encode the month of the year:

$$\text{hour_sin} = \sin\left(2\pi \cdot \frac{\text{hour}}{24}\right), \quad \text{hour_cos} = \cos\left(2\pi \cdot \frac{\text{hour}}{24}\right)$$

$$\text{month_sin} = \sin\left(2\pi \cdot \frac{\text{month}}{12}\right), \quad \text{month_cos} = \cos\left(2\pi \cdot \frac{\text{month}}{12}\right)$$

- **Voltage Mean and Imbalance:**

In raw dataset, we record three voltage metrics: `AC_VOLTAGE_AB`, `AC_VOLTAGE_BC`, and `AC_VOLTAGE_CA`, which represent the three-phase line voltages of the inverter. To help the model better capture the interaction among these voltages, we engineered two additional features:

- `V_mean`: the average of the three-phase voltages

$$V_{\text{mean}} = \frac{V_{AB} + V_{BC} + V_{CA}}{3}$$

- `V_unbalance`: the relative deviation between the maximum and minimum phase voltages

$$V_{\text{unbalance}} = \frac{\max(V_{AB}, V_{BC}, V_{CA}) - \min(V_{AB}, V_{BC}, V_{CA})}{V_{\text{mean}} + \epsilon}$$

where $\epsilon = 10^{-6}$ is a small constant added to avoid division by zero.

These derived features allow the model to capture both the overall voltage level and the degree of asymmetry across the three phases, which are important indicators of inverter stability and potential failure.

- **Temperature Difference (T_{Δ}):**

To capture the thermal stress inside the inverter, we computed the temperature difference between the internal inverter temperature (T_{internal}) and the ambient admission temperature (T_{ambient}):

$$T_{\Delta} = T_{\text{internal}} - T_{\text{ambient}}.$$

This feature reflects the cooling efficiency and heat dissipation condition of the inverter. A larger T_{Δ} indicates that the inverter is operating at a higher internal temperature relative to its surroundings, which is a critical factor in reliability and potential failure prediction.

4.3.7. Data Labeling

Using the identified failure sessions (see Section 4.2), each record in the dataset was assigned one of the following labels: *normal*, *failure*, or *maintenance*.

To enable predictive modeling, we further introduced a prediction horizon t_0 , such that all records occurring within t_0 prior to the start of a failure session were labeled as *to-failure*. In our case, we set $t_0 = 5$ days, indicating that the model is trained to predict inverter breakdowns within a five-day window.

It is important to note that all records falling inside failure sessions or maintenance sessions were excluded from the training data, in order to prevent the model from trivially learning in-session patterns that do not generalize to failure prediction.

4.3.8. Data Split

To evaluate the performance of our model in predicting future failure events, we partitioned the dataset into training, validation, and test sets using a time-based split strategy. Two temporal cut-off points were selected: June 30, 2024 and January 1, 2025.

- Records prior to June 30, 2024 were used as the *training set*.
- Records between June 30, 2024 and January 1, 2025 were used as the *validation set*.
- Records after January 1, 2025 were reserved as the *test set*.

This design ensures that the model is always evaluated on data strictly occurring later in time, which is crucial for realistic failure prediction tasks. The size and label distribution of each split are summarized in Table 3.

Table 3: Summary of dataset splits by time period, size, and label distribution.

Split	Period	Total Records	Normal	To-Failure	% To-Failure
Training	Before June 30, 2024	550,223	546,079	4,144	0.75%
Validation	June 30, 2024 – Dec 31, 2024	141,403	140,683	720	0.51%
Test	After Jan 1, 2025	146,185	145,465	720	0.49%

4.3.9. Standardization

To assess inter-device variability, we first conduct an *ANOVA t-test* on all metrics to examine whether their distributions differ significantly across devices. As shown in Table 4, nearly all selected metrics (e.g., temperatures, voltages, DC voltage, AC power, and engineered features such as T_{δ} and V_{mean}) exhibit highly significant differences across devices ($p < 0.001$). The only exception is $V_{\text{unbalance}}$, whose p -value is close to 1, suggesting that this feature is inherently

normalized and consistent across devices. These results confirm the presence of device-specific biases in the majority of raw features.

To mitigate this issue, we apply **device-wise standardization** to all features listed in Table 4. For each device, the mean and standard deviation are computed independently, and each feature is normalized as

$$x'_{i,j} = \frac{x_{i,j} - \mu_{d(j)}}{\sigma_{d(j)}},$$

where $x_{i,j}$ denotes the value of feature j for device i , and $\mu_{d(j)}$, $\sigma_{d(j)}$ are the mean and standard deviation of feature j computed within the corresponding device d . This procedure effectively removes inter-device scale differences, ensuring that the model learns failure-related patterns rather than device-specific characteristics.

Table 4: ANOVA test results for selected features across devices. Most features show significant device-level differences ($p < 0.001$).

Feature	F-statistic	p-value
STATUS_AC_MOD_ADMISSION_TEMP	174.30	0.0
STATUS_INTERNAL_TEMP	215.60	0.0
AC_VOLTAGE_AB	33.69	4.60×10^{-98}
AC_VOLTAGE_BC	33.69	4.80×10^{-98}
AC_VOLTAGE_CA	33.69	4.75×10^{-98}
DC_VOLTAGE	6949.56	0.0
AC_POWER	6.82	4.91×10^{-15}
V_mean	33.69	4.72×10^{-98}
V_unbalance	0.31	0.995
T_delta	477.55	0.0

4.4. Dataset Construction

After preprocessing, we obtain the cleaned training, validation, and test datasets. To prepare data for modeling, we construct samples using a sliding window approach with a window length of 7 days. Each window is required to be continuous in time, since certain intervals were removed during data cleaning (Section 4.3.5) and labeling (Section 4.3.7).

Formally, given a time series $\{(x_t, y_t)\}_{t=1}^T$, we construct a window $w = \{x_t, x_{t+1}, \dots, x_{t+\Delta-1}\}$ with horizon $\Delta = 336 = 7$ days. The label for each window is defined as the status at the last timestep:

$$y_w = y_{t+\Delta-1}.$$

To address class imbalance, we apply different strides for window generation across splits:

- **Training set:** To increase the number of positive (*to-failure*) samples, we oversample them by using a stride of 2 (i.e., 1 hour).
- **Negative samples and Validation/Test sets:** A stride of 12 (i.e., 6 hours) is used to reduce redundancy and ensure more diverse windows.

This strategy compensates for the severe label imbalance, where less than 1% of records are labeled as *to-failure*, while maintaining a balanced trade-off between sample efficiency and diversity. The dataset size and label distribution after applying the sliding window construction are summarized in Table 5.

Table 5: Summary of constructed dataset using sliding windows (7 days) with different strides.

Split	#Windows	Normal	To-Failure	Percentage of To-Failure
Training	44504	42584	1920	4.31%
Validation	11270	11210	60	0.53%
Test	10979	10919	60	0.55%

4.5. Model Architecture

To capture both the short-term patterns and long-term temporal dependencies in inverter sensor data, we design a hybrid **CNN-LSTM** model. The architecture combines convolutional neural networks (CNNs) for local feature extraction with long short-term memory (LSTM) networks for sequence modeling.

4.5.1. CNN Module

The input to the model is a multivariate time series with 21 features. We first apply a one-dimensional convolutional layer:

$$\text{Conv1D}(21 \rightarrow 32, \text{kernel size} = 3, \text{stride} = 1, \text{padding} = 1),$$

followed by ReLU activation, batch normalization, and dropout ($p = 0.5$). This CNN module captures short-term dependencies and local fluctuations across neighboring time steps, while also normalizing feature distributions and preventing overfitting.

4.5.2. LSTM Module

The CNN output is then passed into a two-layer LSTM with hidden size 64:

$$\text{LSTM}(32 \rightarrow 64, \text{ layers} = 2, \text{ dropout} = 0.5).$$

The LSTM module models long-term dependencies and sequential dynamics in the inverter signals, which are crucial for identifying gradual degradation patterns leading to failure.

4.5.3. Classifier

The final hidden representation from the LSTM is fed into a fully connected classifier:

$$64 \rightarrow 32 \rightarrow 1,$$

with ReLU activation and dropout ($p = 0.5$) applied after the first linear layer. The output layer produces a single scalar probability via sigmoid activation (applied during inference), representing the likelihood that the input window belongs to a *to-failure* class.

4.5.4. Overall Architecture

In summary, the CNN-LSTM architecture, shown in Figure 5, consists of:

- **CNN module:** 1D convolution + ReLU + BatchNorm + Dropout,
- **LSTM module:** 2-layer LSTM with hidden size 64,
- **Classifier:** two fully connected layers ($64 \rightarrow 32 \rightarrow 1$) with dropout and ReLU.

This design allows the model to learn both short-term fluctuations and long-term dependencies, providing a robust foundation for predicting inverter failure within the specified horizon.

4.6. Training Procedure

Given the highly imbalanced nature of our dataset, where “to-failure” samples account for less than 1% of the total records, we adopt the **Focal Loss** function [32] as our training objective. Focal Loss introduces a modulating factor to the standard cross-entropy loss in order to focus learning on hard misclassified examples, which is particularly effective under severe class imbalance.

performance, using AUCPR, Precision@K, and Recall@K as evaluation metrics. All experiments are conducted on an NVIDIA RTX 2060 GPU, with each training run requiring less than ten minutes. The best-performing checkpoint on the validation set is retained for testing and will be further evaluated in the next section.

5. Analysis

5.1. Evaluation methodology

We train on data before June 30, 2024, tune on June 30–Dec 31, 2024, and test on Jan 1, 2025 and later. After we turn the raw time series into 7-day windows with a 6-hour stride, failures are rare: about 0.0053 of windows on validation (60 of 11,270) and 0.0055 on test (60 of 10,979). With such imbalance, accuracy can be misleading, since a model that always says normal would be 99.5% accurate in this case. Therefore, we evaluate with precision, recall, and ROC AUC.

- Precision answers “when we raise an alert, how often are we right?” With 0.5% positives, even a small absolute precision (e.g., 5–6%) can be a big lift over random (0.5%).
- Recall answers “of all true failures, how many did we catch?”
- ROC AUC summarizes how well the model ranks failure windows above normal ones across all thresholds. ROC is less sensitive to class imbalance than accuracy, so it’s good for checking overall discrimination.

5.2. Score calibration and thresholding

To turn raw model scores into well-behaved probabilities and pick an operating point, we calibrate with logistic regression on the validation set. We fit a 1D logistic model on the model’s logits vs. the true labels from June 30–Dec 31, 2024. This yields calibrated probabilities that better match the observed base rate. We then choose a decision threshold on the validation precision–recall curve to target a desired recall level (e.g., 0.80) and report the corresponding precision and recall on validation.

For the test period after Jan 1, 2025, we use a fixed threshold of 0.5. Because the score distribution and class balance shifted in 2025, the validation derived threshold no longer transfers on precision/recall and ROC AUC drop. Thus, we recommend re-calibrating on the most recent labeled slice before deployment and re-selecting the threshold to match your operational target.

Given the clear temporal shift, we also recommend periodic retraining so the model and its calibration stay aligned with 2025 conditions.

5.3. Main results

On training dataset, the model recalls most failures (recall = 0.86 for the Failure class) with lower precision (0.26), and ROC AUC = 0.94. This shows that the model performs quite well on seen data.

Table 6: Training classification report

Class	Precision	Recall	F1-score	Support
Normal	0.99	0.89	0.94	42,584
Failure	0.26	0.86	0.40	1,920
Accuracy		0.89		44,504
Macro avg	0.63	0.87	0.67	44,504
Weighted avg	0.96	0.89	0.92	44,504
ROC AUC		0.9380		

Table 7: Training confusion matrix

	Pred: Normal	Pred: Failure
True Normal	37,974	4,610
True Failure	277	1,643

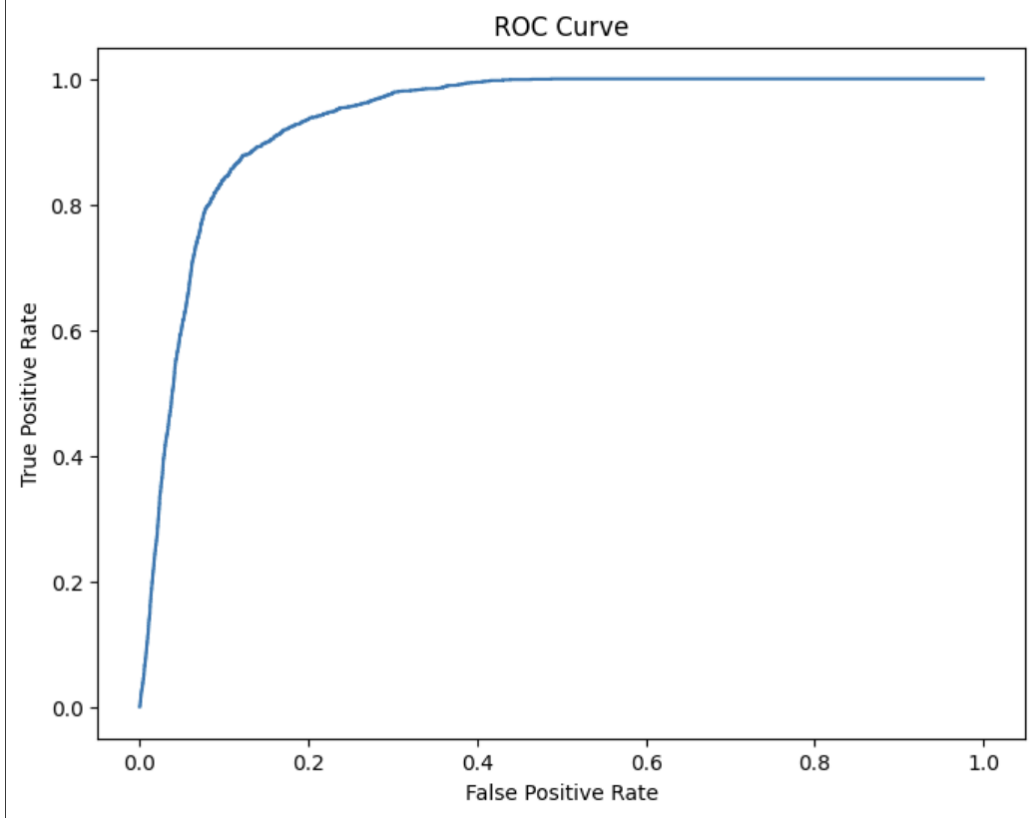


Figure 6: ROC curve for training set.

On validation, after calibration and choosing a threshold to target 0.80 recall, we reach recall = 0.80 and precision = 0.05 with ROC AUC = 0.95. Note that 0.05 precision is 10 times larger the base rate (0.51%), so while there are still false alarms, the alerts are far more concentrated than chance.

Table 8: Validation classification report

Class	Precision	Recall	F1-score	Support
Normal	1.00	0.92	0.96	11,210
Failure	0.05	0.80	0.09	60
Accuracy		0.92		11,270
Macro avg	0.52	0.86	0.53	11,270
Weighted avg	0.99	0.92	0.95	11,270
ROC AUC		0.9535		

Table 9: Validation confusion matrix

	Pred: Normal	Pred: Failure
True Normal	10,299	911
True Failure	12	48

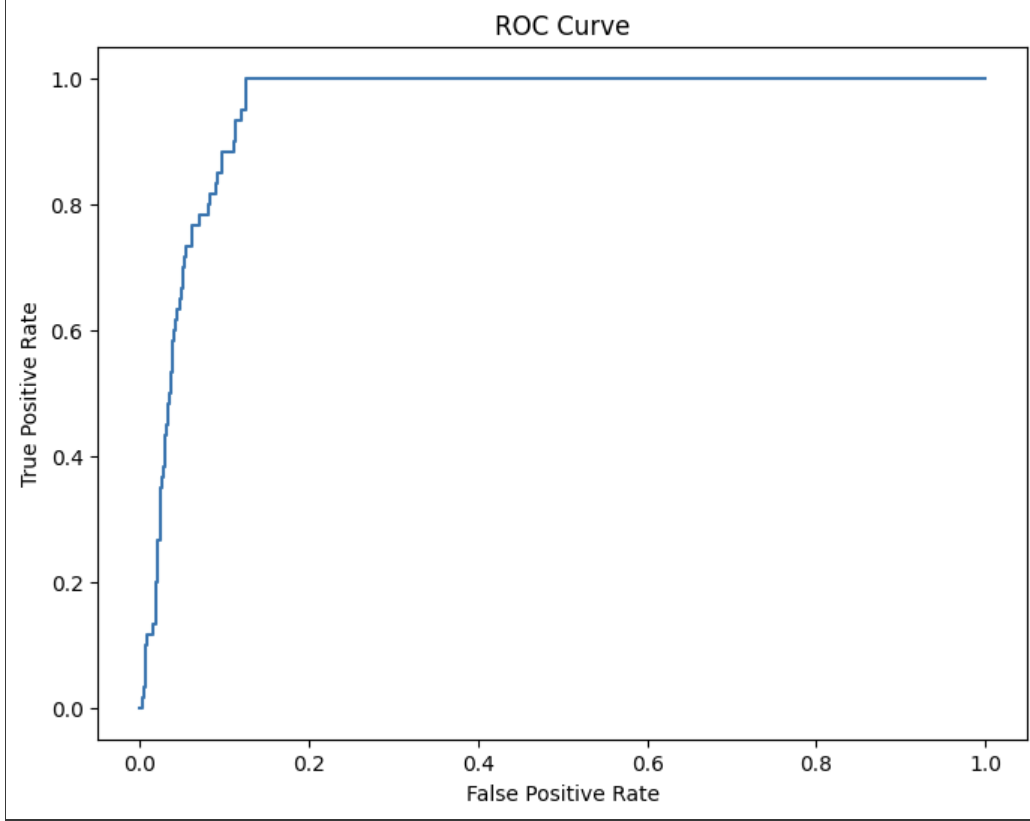


Figure 7: ROC curve for validation set.

With a fixed 0.5 cutoff, we observe a recall of 0.11, precision of approximately 0.02, and ROC AUC of about 0.69 (see Table 10 and Figure 8). Since $\text{AUC} > 0.5$, our model still ranks failure windows above normal ones more often than chance. In practice, there are several ways to improve. We suggest refitting calibration on the most recent labeled slice, or retraining on a recent window to keep up with 2025 drift.

Table 10: Test classification report

Class	Precision	Recall	F1-score	Support
Normal	0.99	0.94	0.97	32,583
Failure	0.02	0.11	0.03	300
Accuracy		0.94		32,883
Macro avg	0.50	0.53	0.50	32,883
Weighted avg	0.98	0.94	0.96	32,883
ROC AUC		0.6908		

Table 11: Test confusion matrix

	Pred: Normal	Pred: Failure
True Normal	30,714	1,869
True Failure	266	34

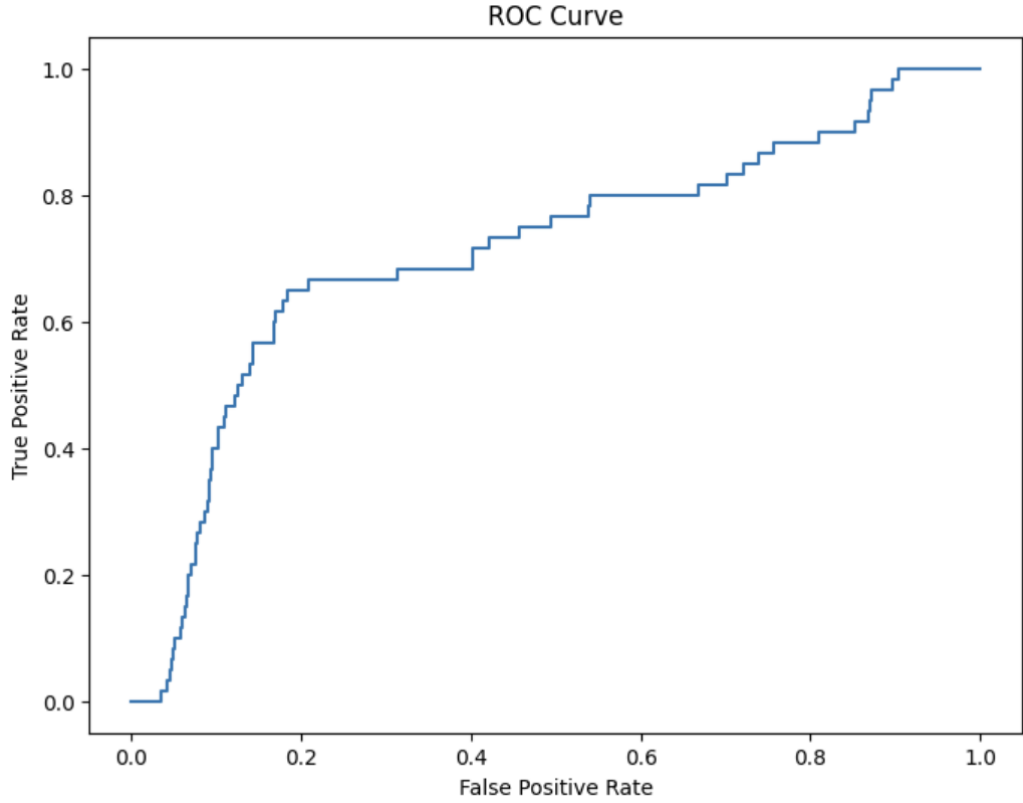


Figure 8: ROC curve for test set.

5.4. Post-processing methodology

The model first produces a risk score for each time window. We turn those scores into provisional alerts using the probability cut-off chosen on recent validation data. Next, we apply a persistence check: an alert is only kept if the same device stays positive across three consecutive windows, which filters out one-off spikes. Finally, we merge each kept run into a single incident with start/end times and evaluate performance on these cleaned alerts. This makes alerts more stable and more actionable without changing the underlying model.

5.5. Effect of temporal smoothing

After applying the temporal smoothing, the validation set kept 89 sustained runs and filtered out 124 short blips. At the calibrated cut-off, validation now lands at about 0.06 precision and

0.77 recall with ROC AUC: 0.95. In other words, smoothing removes noisy one-offs and nudges precision up a bit while recall drops a little, which is exactly what we expect when we require persistence.

Table 12: Validation classification report (smoothed, calibrated). Kept: 89; removed: 124.

Class	Precision	Recall	F1-score	Support
Normal	1.00	0.93	0.97	11,210
Failure	0.06	0.77	0.11	60
Accuracy		0.93		11,270
Macro avg	0.53	0.85	0.54	11,270
Weighted avg	0.99	0.93	0.96	11,270
ROC AUC		0.9535		

Table 13: Validation confusion matrix (smoothed, calibrated)

	Pred: Normal	Pred: Failure
True Normal	10,466	744
True Failure	14	46

On the test set, applying a 3-window temporal smoothing per device kept 197 runs and removed 57 short blips. Using the same calibrated scores and a fixed 0.5 cutoff, we get recall = 0.10, precision = 0.02, and ROC AUC = 0.6908 (see Tables 14, Fig. 9). Smoothing trims isolated spikes, but it does not overcome the score shift from late 2024 to 2025. Many true failures still sit below 0.5.

Table 14: Test classification report (smoothed, calibrated). Kept: 197; removed: 57.

Class	Precision	Recall	F1-score	Support
Normal	0.99	0.94	0.97	32,583
Failure	0.02	0.10	0.03	300
Accuracy		0.94		32,883
Macro avg	0.50	0.52	0.50	32,883
Weighted avg	0.98	0.94	0.96	32,883
ROC AUC		0.6908		

Table 15: Test confusion matrix (smoothed, calibrated)

	Pred: Normal	Pred: Failure
True Normal	30,767	1,816
True Failure	270	30

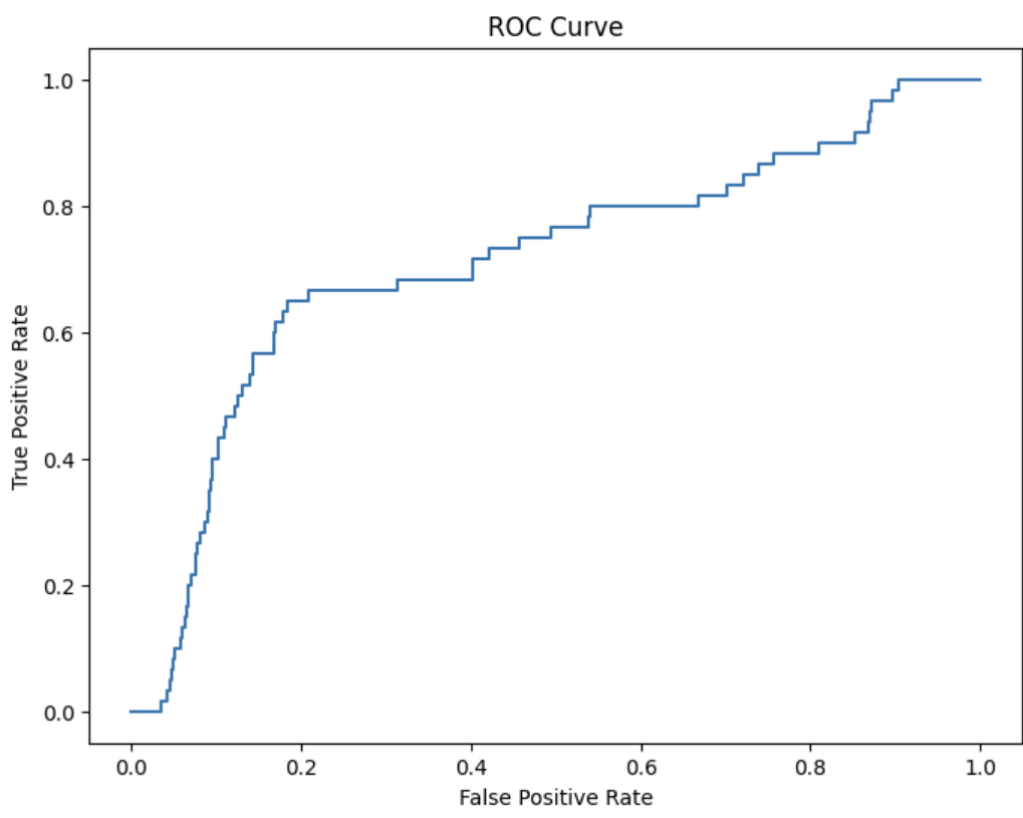


Figure 9: ROC curve for test set (smoothed, calibrated).

6. Conclusion and Future Work

6.1. Conclusion

In this project, we addressed the challenge of predictive maintenance for large-scale inverters by developing a hybrid deep learning framework that integrates convolutional neural networks (CNNs) and long short-term memory (LSTM) architectures. The research began with a comprehensive data transformation pipeline, converting more than three million inverter records into a structured time-series dataset suitable for forecasting. Through extensive preprocessing like removal of anomalous intervals, imputation of missing values, device-wise standardization, down-sampling to reduce noise, and feature engineering for cyclic temporal patterns, voltage imbalance, and thermal stress which the raw signals were transformed into a high-quality dataset capable of supporting robust failure prediction. The use of failure session labeling and a five-day prediction horizon enabled the model to focus specifically on early indicators of degradation, rather than post-failure signatures, and then align the modeling task with real-world maintenance objectives.

The CNN-LSTM design proved well-suited to this task. The CNN layers learned short-term

patterns in the data and helped smooth out small fluctuations and noise. The LSTM layers then captured longer-term trends that are important for spotting gradual signs of failure. This architecture was paired with focal loss to mitigate severe class imbalance, which helps the model pay more attention to these rare but important events instead of being dominated by the large number of normal operation records. Training with seven-day sliding windows showed potential in improving the model’s ability to capture early signs of failure, while evaluation with ROC AUC suggested that the model could differentiate between normal and failure cases reasonably well. These results indicate that the proposed approach may provide operators with early warnings of inverter breakdowns up to around five days in advance, offering a possible lead time that could help reduce downtime, limit energy loss, and improve planning for maintenance activities.

6.2. Limitations and Future Work

Nevertheless, several limitations of the present study must be acknowledged. The rarity of inverter failure events poses challenges for generalization, even with the use of focal loss and oversampling strategies. At present, the model relies solely on inverter data. Incorporating additional data such as weather, irradiance, and detailed event logs could help separate real risk from normal swings and improve predictive accuracy. Because operating conditions change over time, regular score recalibration and periodic retraining are recommended to maintain model reliability. While the current CNN–LSTM framework performed well on the available dataset, more advanced architectures such as Transformer-based models may further enhance the ability to capture long-sequence dependencies and improve interpretability. Despite these challenges, the model has shown meaningful ability to rank risky periods, and after calibration with temporal smoothing, our model produces alerts that are much more usable for ops while keeping good coverage of true problems.

Looking forward, this approach could be extended within MN8 Energy and similar green energy companies to build broader predictive maintenance systems. Beyond inverters, a comparable framework could be applied to other critical assets, such as transformers, batteries, or monitoring equipment to estimate the likelihood and timing of failures. By tracking emerging risk patterns and degradation trends, such models could support the planning of regular inspections and preventive maintenance and help operators schedule work proactively rather than reactively. In practice, this would not only reduce unplanned downtime but also improve resource allocation and ensure that maintenance crews and spare parts are deployed where and when they are most

needed. Over time, integrating these predictive insights into operational workflows could enhance the overall reliability and cost-effectiveness of large-scale solar portfolios.

7. Acknowledgments

We would like to express our sincere gratitude to MN8 Energy, specifically Ramon Millan for their continuous support throughout this project. Their provision of inverter data, participation in multiple project discussions, and constructive guidance on implementation directions and modeling objectives were invaluable in shaping our research. We also extend our gratitude to the UCLA Samueli School of Engineering Master of Engineering Program for offering academic and administrative support that enabled us to successfully carry out this capstone project. Finally, we are grateful to Professors Bruce Huang and Joey Lao for their mentorship, coordination, and ongoing feedback, which greatly contributed to the development and completion of our work.

References

- [1] K. Eurek, C. Murphy, W. Cole, W. Frazier, P. Brown, and A. Schleifer, “Representing dc-coupled pv+battery hybrids in a capacity expansion model,” 2021.
- [2] N. DiOrio, P. Denholm, and W. Hobbs, “A model for evaluating the configuration and dispatch of pv plus battery power plants,” *Applied Energy*, vol. 262, 2020, publisher Copyright: © 2020 Elsevier Ltd.
- [3] S. Ericson, S. Koebrich, S. Awara, A. Schleifer, J. Heeter, K. Cory, C. Murphy, and P. Denholm, “Influence of hybridization on the capacity value of pv and battery resources,” 2022.
- [4] R. Dufo-López, J. M. Lujano-Rojas, J. S. Artal-Sevil, and J. L. Bernal-Agustín, “Optimising grid-connected pv-battery systems for energy arbitrage and frequency containment reserve,” *Batteries*, vol. 10, no. 10, p. 427, 2024.
- [5] J. He, Y. Yang, and D. Vinnikov, “Energy storage for 1500 v photovoltaic systems: A comparative reliability analysis of dc- and ac-coupling,” *Energies*, vol. 13, no. 13, p. 3355, 2020.
- [6] V. Dunna, K. Chandra, P. Rout *et al.*, “Super-twisting mppt control for grid-connected pv/battery system using higher order sliding mode observer,” *Scientific Reports*, vol. 14, no. 16597, 2024.
- [7] R. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.
- [8] R. Rajagukguk, R. Ramadhan, and H.-J. Lee, “A review on deep learning models for forecasting time series data of solar irradiance and photovoltaic power,” *Energies*, vol. 13, no. 24, p. 6623, 2020.
- [9] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.
- [10] G. Reikard, “Predicting solar radiation at high resolutions: A comparison of time series forecasts,” *Solar Energy*, vol. 83, no. 3, pp. 342–349, 2009.
- [11] R. J. Hyndman and G. Athanasopoulos, *Forecasting: principles and practice*, 3rd ed. Melbourne, Australia: OTexts, 2022.
- [12] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [13] B. Lim and S. Zohren, “Time-series forecasting with deep learning: A survey,” *Philosophical Transactions of the Royal Society A*, vol. 379, no. 2194, p. 20200209, 2021.
- [14] S. M. Miraftebadeh *et al.*, “Data anomaly detection in photovoltaic power time-series using a deep autoencoder,” *Renewable Energy Reports*, 2025.
- [15] T. Park *et al.*, “Convolutional autoencoder-based anomaly detection in pv forecasting to improve data integrity and model accuracy,” *Energies*, vol. 16, no. 14, p. 5293, 2023.
- [16] A. Dairi, F. Harrou, Y. Sun, and S. Khadraoui, “Short-term forecasting of photovoltaic solar power production using variational auto-encoder driven deep learning,” *Applied Sciences*, vol. 10, no. 23, p. 8400, 2020.
- [17] X. Hou *et al.*, “Prediction of solar irradiance using convolutional neural network and attention-based lstm,” *Journal of Cleaner Energy Forecasting*, 2023.

- [18] S. C. Lim *et al.*, “Solar power forecasting using a cnn–lstm hybrid model,” *Energies*, vol. 15, no. 21, p. 8233, 2022.
- [19] A. P. Wibawa, “Time-series analysis with smoothed convolutional neural networks,” *Journal of Big Data*, 2022.
- [20] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [21] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [22] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [23] F. Wang *et al.*, “Solar irradiance short-term forecasting using lstm networks,” *Energies*, vol. 12, no. 13, p. 2571, 2019.
- [24] J. Bedi and D. Toshniwal, “Deep learning framework to forecast electricity demand,” *Applied Energy*, vol. 238, pp. 131–146, 2020.
- [25] K. Cho *et al.*, “Learning phrase representations using rnn encoder–decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
- [26] J. Zhang, J. Yan, and D. Infield, “Short-term forecasting of wind power using a novel hybrid model based on gru,” *Energies*, vol. 12, no. 6, p. 1044, 2019.
- [27] F. Shahid, A. Zameer, and M. Muneeb, “A novel deep learning model for short-term solar power forecasting,” *IEEE Access*, vol. 9, pp. 18 124–18 134, 2021.
- [28] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [29] H. Wu *et al.*, “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, 2020, pp. 11 106–11 115.
- [30] B. Lim, S. Ö. Arik, N. Loeff, and T. Pfister, “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.
- [31] M. Q. Raza *et al.*, “Hybrid deep learning models for renewable energy forecasting: A review,” *Renewable and Sustainable Energy Reviews*, vol. 156, p. 111978, 2022.
- [32] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 2, pp. 318–327, 2020.

A. Appendix

A.1. Raw Data

Table 16: Raw data

Column	unit	example
device_id	N/A	2009.21.001.BLK02.PAD03.INV01
device_name	N/A	INV 57
device_type	N/A	Inverter
site_id	N/A	2009.21.001
site_name	N/A	Slate→†BART
site_latitude	°	36.226944
site_longitude	°	-119.893889
site_ac_capacity	W	50496
event_utc_time	ISO 8601 UTC timestamp	2025-06-25T07:00:00.000Z
event_local_time	ISO 8601 UTC timestamp	2025-06-25T00:00:00.000Z
metric	N/A	AC_POWER.MEASURED
value	N/A	0

A.2. Metrics List

Table 17: Current-related metrics

Metric	Note	Unit	Type
AC_CURRENT_A	AC current phase A	A	float
AC_CURRENT_B	AC current phase B	A	float
AC_CURRENT_C	AC current phase C	A	float
AC_CURRENT_MAX	Max AC current	A	float
DC_CURRENT	DC current	A	float
DC_CURRENT_AVG	Avg DC current	A	float
DC_CURRENT_MAX	Max DC current	A	float
STATUS_CURRENT_NORMATIVE	Current normative status	A	categorical

Table 18: Electrical metrics (general)

Metric	Note	Unit	Type
FREQUENCY	Grid frequency	Hz	float

Table 19: Energy-related metrics

Metric	Note	Unit	Type
ENERGY_DELIVERED	Cumulative delivered energy	kWh	float
ENERGY_DELIVERED_DAILY	Daily delivered energy	kWh/day	float
ENERGY_DELIVERED_MONTHLY	Monthly delivered energy	kWh/month	float
ENERGY_RECEIVED	Cumulative received energy	kWh	float
VARH_DELIVERED	Reactive energy (cumulative)	kvarh	float
VARH_DELIVERED_DAILY	Reactive energy (daily)	kvarh/day	float
VARH_DELIVERED_MONTHLY	Reactive energy (monthly)	kvarh/month	float

Table 20: Insulation monitoring metrics

Metric	Note	Unit	Type
INSUL_MON_AC_RESISTOR	AC insulation monitor (resistance)	-	float
INSUL_MON_DC_RESISTOR	DC insulation monitor (resistance)	-	float
STATUS_INSUL_MON_AC	AC insulation monitor	-	float
STATUS_INSUL_MON_DC	DC insulation monitor	-	float

Table 21: Miscellaneous metrics

Metric	Note	Unit	Type
COMM_LINK	Communication link quality/status	-	float
HEARTBEAT	Heartbeat/keep-alive	-	float
HW_VERSION	Hardware version	-	string

Table 22: Power-related metrics

Metric	Note	Unit	Type
AC_POWER	Active power (AC)	kW	float
AC_POWER_LIMIT_SETPOINT	Setpoint: AC power limit	%	float
DC_POWER	DC power	kW	float
POWER_FACTOR	Power factor	-	float
STATUS_POWER_SOURCE_1	Power source 1 status	kW	categorical
STATUS_POWER_SOURCE_2	Power source 2 status	kW	categorical
SVA	Apparent power	kVA	float
SVA_LIMIT_SETPOINT	Setpoint: apparent power limit	%	float
VAR	Reactive power	kvar	float
VAR_LIMIT_SETPOINT	Setpoint: reactive power limit	%	float

Table 23: Status metrics

Metric	Note	Unit	Type
STATUS_AC_MOD_ADMISSION_TEMP	Ambient/module admission temperature	°C	float
STATUS_FAULT_MODULE	Fault status (module)	-	integer (code/bitmask)
STATUS_FAULT_WORD	Fault word/code	-	integer (code/bitmask)
STATUS_IGBT_MAX_TEMP	IGBT max temperature	°C	float
STATUS_INTERNAL_HUMIDITY	Internal humidity	%	float
STATUS_INTERNAL_INPUT_WORD	Internal input word	-	integer (code/bitmask)
STATUS_INTERNAL_OUTPUT_WORD	Internal output word	-	integer (code/bitmask)
STATUS_INTERNAL_TEMP	Internal inverter temperature	°C	float
STATUS_LV_PRESSURE	Low-voltage pressure/status	-	float
STATUS_MOD_MAX_TEMP	Module max temperature	°C	float
STATUS_MV_PRESSURE	Medium-voltage pressure/status	-	float
STATUS_WARNING_MODULE	Warning status (module)	-	integer (code/bitmask)
STATUS_WARNING_WORD	Warning word/code	-	integer (code/bitmask)
STATUS_WORD	Status word	-	integer (code/bitmask)

Table 24: Voltage-related metrics

Metric	Note	Unit	Type
AC_VOLTAGE_AB	Line voltage AB	V	float
AC_VOLTAGE_BC	Line voltage BC	V	float
AC_VOLTAGE_CA	Line voltage CA	V	float
AC_VOLTAGE_HI_SETPOINT	Setpoint: AC voltage high limit	%	float
AC_VOLTAGE_LO_SETPOINT	Setpoint: AC voltage low limit	%	float
DC_BATT_VOLTAGE_BUS	Battery DC bus voltage	V	float
DC_VOLTAGE	DC voltage	V	float
DC_VOLTAGE_BUS	DC bus voltage	V	float
DC_VOLTAGE_N	DC negative rail voltage	V	float
DC_VOLTAGE_P	DC positive rail voltage	V	float